Multilayer Perceptrons: Architecture, Training, and Real-World Applications

Author
VRUSHALI PATEL

1. Introduction

Overview of Machine Learning

Machine learning (ML) is a branch of artificial intelligence that enables computers to learn patterns from data and make predictions or decisions without being explicitly programmed for each task. Broadly, ML is divided into supervised learning, unsupervised learning, and reinforcement learning. In supervised learning, a model is trained on labelled input-output pairs, making it suitable for tasks such as classification and regression.

What is a Neural Network?

A neural network is a computational model inspired by the structure of the human brain. It consists of interconnected nodes (neurons) organised in layers that process information by passing signals through weighted connections. Neural networks are particularly effective at learning complex, non-linear relationships within data, making them powerful tools for image recognition, natural language processing, and predictive analytics.

What is a Multilayer Perceptron (MLP)?

A Multilayer Perceptron (MLP) is one of the most fundamental types of artificial neural network. It consists of at least three layers of nodes: an input layer, one or more hidden layers, and an output layer. Unlike a simple single-layer perceptron, the MLP can model non-linear decision boundaries, making it far more versatile. MLPs are trained using backpropagation and are widely used across classification, regression, and pattern recognition tasks.

2. MLP Architecture

Input, Hidden, and Output Layers

An MLP is organised into three principal layer types:
Input Layer: Receives the raw feature data. Each neuron in this layer represents one input feature (e.g. blood pressure, age).
Hidden Layers: One or more intermediate layers that apply weighted transformations and activation functions to learn abstract representations of the data.

Output Layer: Produces the final prediction. For binary classification, a single neuron with sigmoid activation is used; for multi-class problems, softmax is applied.

Fully Connected Structure & Architecture Diagram

In an MLP, every neuron in one layer is connected to every neuron in the next — this is a fully connected or dense architecture. Each connection carries a learnable weight. The diagram below illustrates a two-hidden-layer MLP with 3 inputs and 1 output:

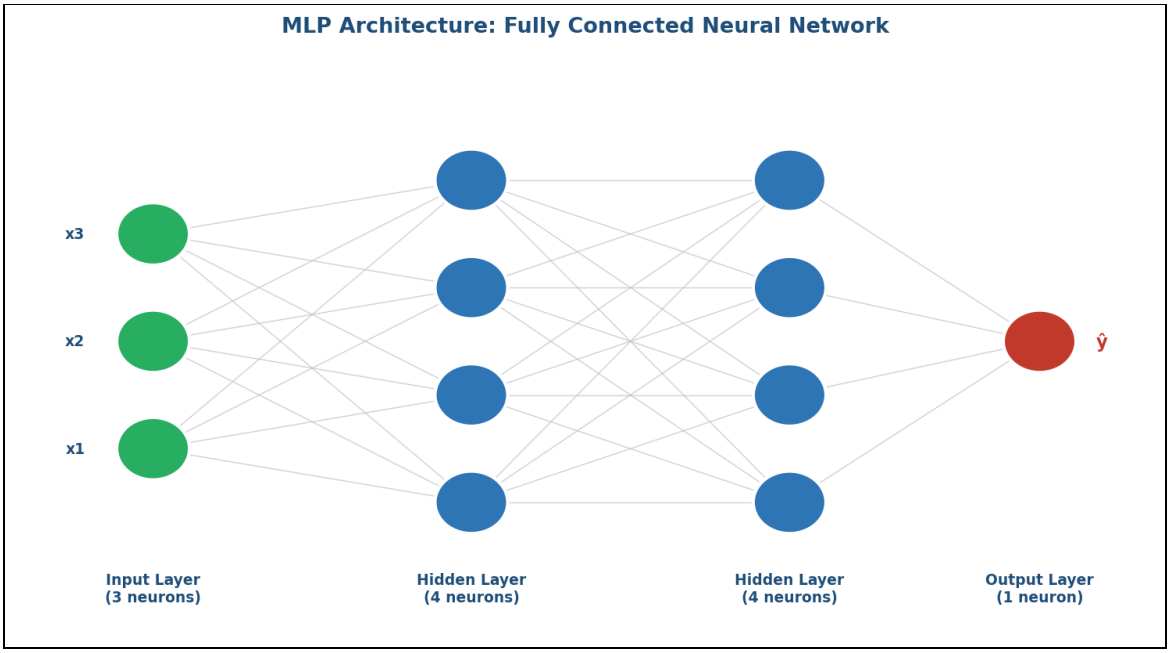

Fig 1: MLP architecture showing input, two hidden layers, and output layer with fully connected neurons

3. Core Concepts

Weights and Bias

Weights (w) are learnable parameters that determine the strength of the connection between neurons. A bias (b) is an additional parameter added to each neuron, allowing the activation function to shift. Together, the linear transformation at each neuron is: $z = w \cdot x + b$, where x is the input vector.

Activation Functions

Activation functions introduce non-linearity into the network, enabling it to learn complex patterns. The two most common are ReLU and Sigmoid, plotted below:

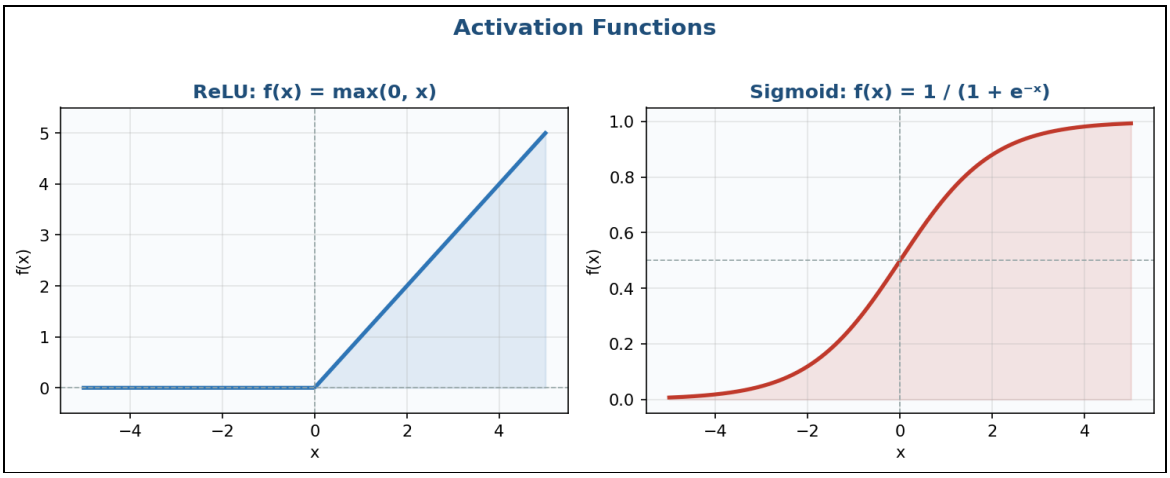

Fig 2: ReLU (left) outputs zero for negative inputs; Sigmoid (right) maps all values to (0, 1)

Forward Propagation

Forward propagation is the process by which input data is passed through the network layer by layer to generate a prediction. At each layer, the weighted sum of inputs is computed ($z = w \cdot x + b$), an activation function is applied, and the result is passed to the next layer. The diagram below summarises this data flow:

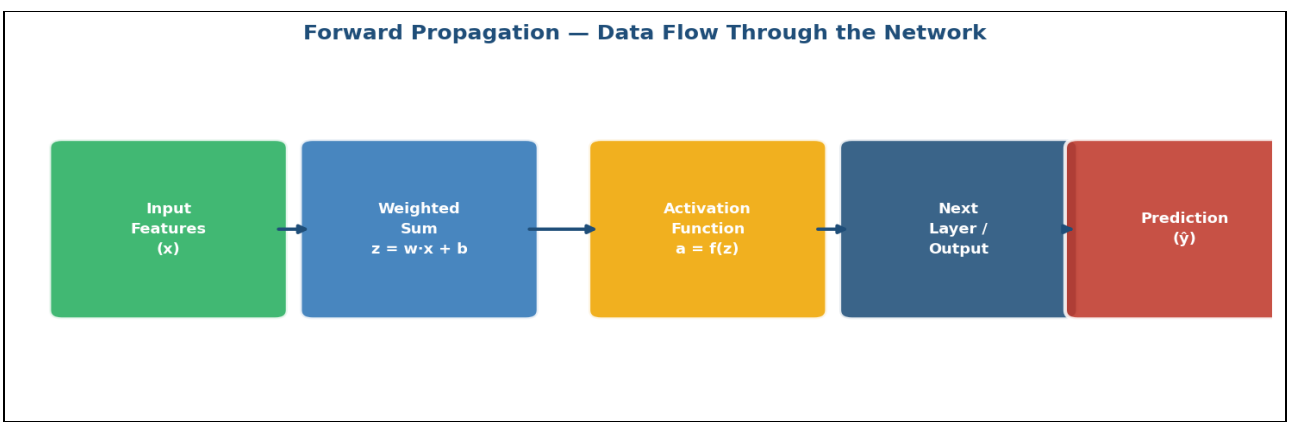

Fig 3: Forward propagation — data flows from inputs through weighted sums and activations to the final prediction

4. Training the Model

Loss Function

A loss function measures the difference between the model's predictions and the actual target values. For binary classification, Binary Cross-Entropy is commonly used; for regression tasks, Mean Squared Error (MSE) is standard. The goal of training is to minimise this loss.

Backpropagation

Backpropagation is the algorithm used to train an MLP. After forward propagation produces a prediction, the error is computed using the loss function. This error is then propagated backwards through the network, and the contribution of each weight to the error is calculated using the chain rule of calculus. The weights are subsequently updated to reduce the error.

Gradient Descent

Gradient descent is the optimisation algorithm used to update weights during training. It computes the gradient of the loss with respect to each weight and moves the weights in the direction that reduces the loss. The step size is controlled by the learning rate (α). The visualisation below shows gradient descent converging toward the minimum of a loss surface:

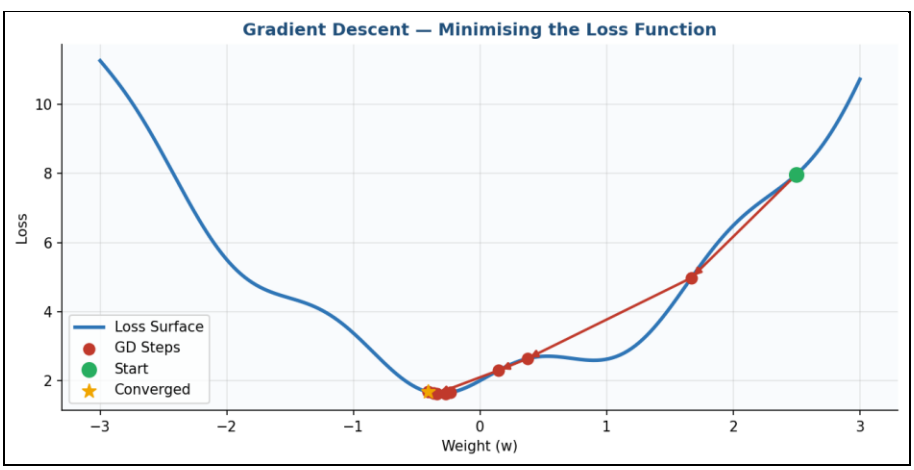

Fig 4: Gradient descent — each step moves weights along the loss surface toward the minimum

Train/Test Split

Before training, the dataset is divided into a training set (typically 70–80%) and a test set (20–30%). The model learns exclusively from the training data, while the test set evaluates generalisation. This split is critical for detecting overfitting.

5. Implementation Example

Dataset

For this example, a synthetic dataset was generated to simulate a loan approval system. The dataset contains 200 records with the following features:

- Income
- Credit Score
- Loan Amount
- Employment Years

The target variable is **Loan Approval**, where:

- 1 = Approved
- 0 = Rejected

The dataset was designed with logical conditions (e.g., higher income and credit score increase approval likelihood), allowing the model to learn meaningful patterns.

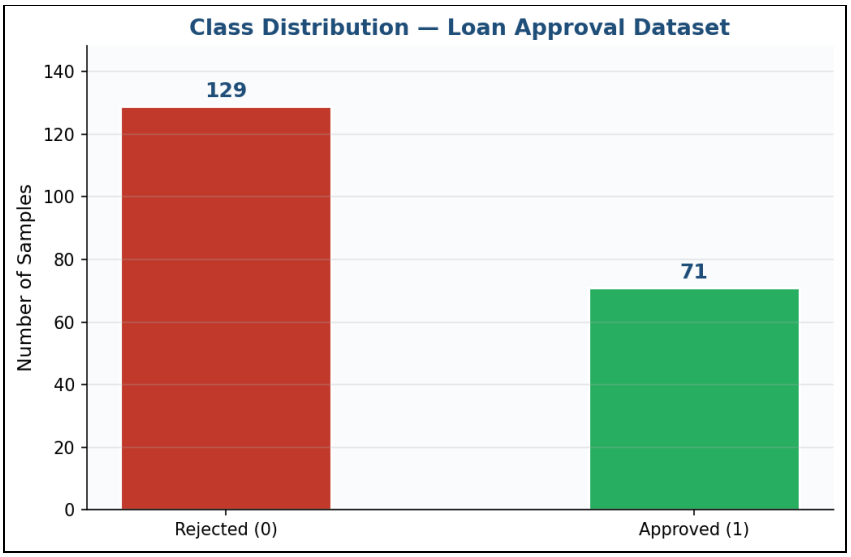

Fig 5: Class distribution showing counts of Rejected (0) and Approved (1) applications

Data Preprocessing

Before training the model, the dataset was prepared as follows:

- The features (X) and target variable (y) were separated
- The dataset was split into training and testing sets using an 80:20 ratio
- Feature scaling was applied using **StandardScaler**, which standardises the data to have zero mean and unit variance

This step is important because neural networks are sensitive to the scale of input features.

Building the MLP Model (Python)

The Multilayer Perceptron (MLP) model was implemented using the Scikit-learn library as shown below:

```
from sklearn.neural_network import MLPClassifier

mlp = MLPClassifier(
    hidden_layer_sizes=(16, 8),
    activation='relu',
    solver='adam',
    max_iter=1000,
    random_state=42
)
```

The model consists of:

- Two hidden layers with 16 and 8 neurons
- ReLU activation function for non-linearity
- Adam optimizer for efficient training

Training and Prediction

The model was trained on the training dataset and then used to make predictions on the test dataset.

```
mlp.fit(X_train, y_train)
y_pred = mlp.predict(X_test)
```

Additionally, the model was tested on new input data to simulate real-world loan approval decisions.

6. Results and Evaluation

Accuracy and Performance

The trained model achieved an accuracy of **92.5%** on the test dataset, indicating strong predictive performance.

Confusion Matrix

```
[[27  0]
 [ 3 10]]
```

Interpretation

- 27 loans were correctly predicted as rejected
- 10 loans were correctly predicted as approved
- 3 approved loans were incorrectly predicted as rejected
- 0 rejected loans were incorrectly predicted as approved

This shows that the model is highly reliable, especially in avoiding incorrect approvals.

Classification Metrics

- **Precision (Approved): 1.00**
  All approved predictions were correct
- **Recall (Approved): 0.77**
  Some approved cases were missed
- **F1-Score: 0.87**
  Indicates a good balance between precision and recall

Loss Curve Analysis

The loss curve shows a steady decrease over training iterations, indicating that the model is learning effectively and converging without instability.

Short Interpretation

An accuracy of 92.5% means the model correctly predicts loan approval decisions for most applicants. The high precision ensures that approved loans are reliable, which is important in financial applications. However, some approved cases are missed, suggesting room for improvement in recall.

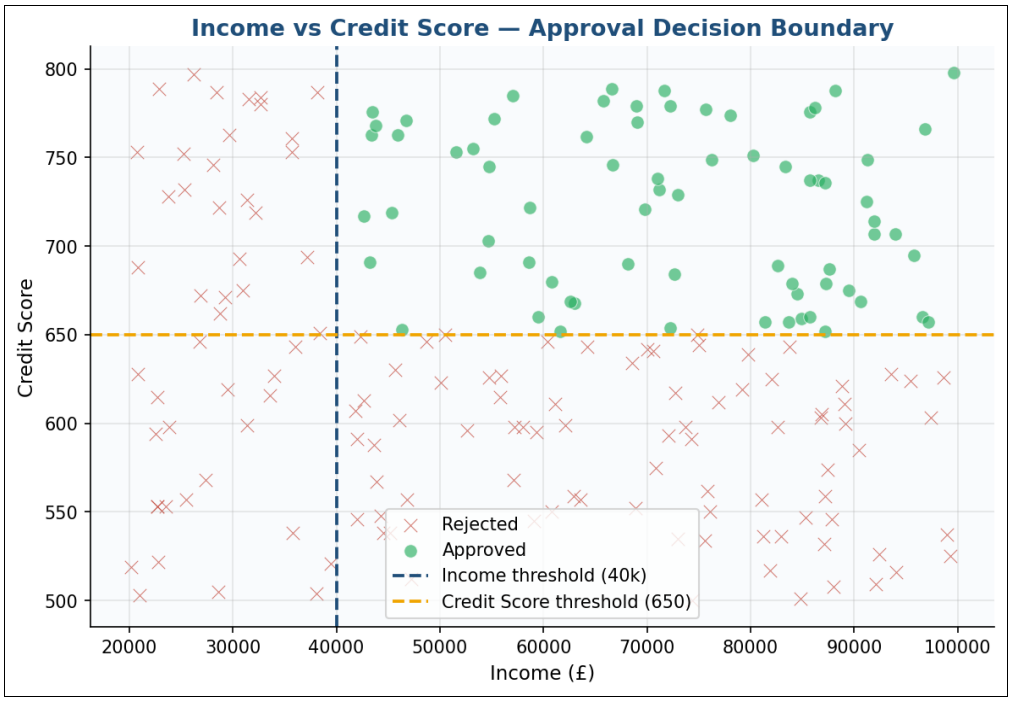

Fig 6: Income vs Credit Score scatter plot — dashed lines mark the decision thresholds at £40,000 income and 650 credit score

7. Overfitting and Improvement

Overfitting

Overfitting occurs when the model learns the training data too well — including its noise — resulting in poor generalisation to unseen data. The plot below illustrates the classic divergence between training and validation loss that signals overfitting:

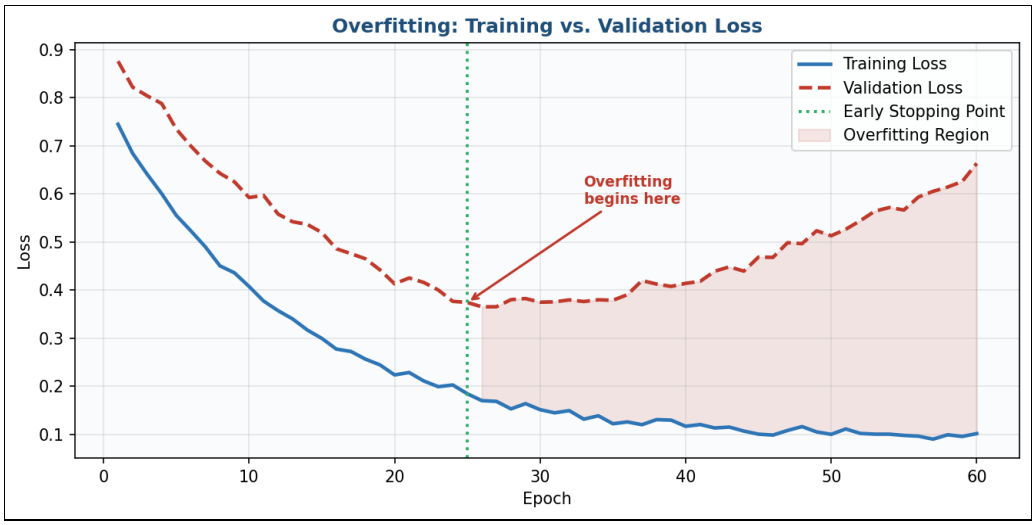

Fig 7: After the early stopping point (~epoch 25), validation loss rises while training loss continues to fall — a clear sign of overfitting

Regularisation Techniques

Dropout: Randomly deactivates a proportion of neurons during each training step (e.g. 30%), forcing the network to learn more robust, distributed representations.

L2 Regularisation (Weight Decay): Adds a penalty proportional to the square of the weights to the loss function, discouraging excessively large weights.

Early Stopping: Monitors validation loss and halts training when it begins to increase, preventing overfitting without manual tuning.

8. Applications

MLPs are deployed across many high-impact domains. The chart below summarises typical accuracy levels achieved by MLP-based solutions in key application areas:

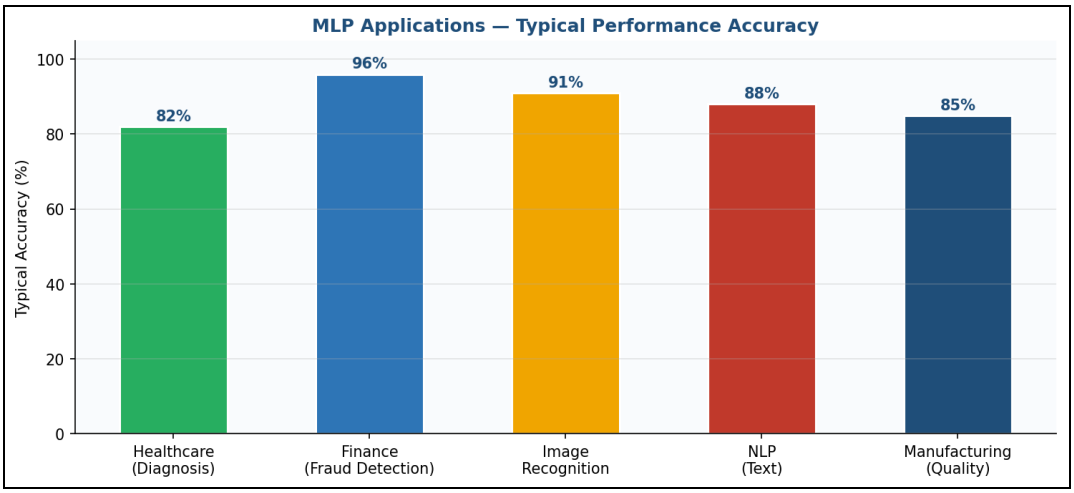

Fig 8: Typical MLP accuracy across five major application domains

Healthcare

MLPs are used in medical diagnosis, drug discovery, and patient outcome prediction. Applications include predicting diabetes risk, detecting tumours from patient records, and assisting in personalised treatment planning.

Finance

In the financial sector, MLPs power credit scoring systems, fraud detection, and stock price forecasting. By learning complex non-linear relationships between economic indicators, they outperform traditional linear models in many scenarios.

Image Recognition

Though CNNs are dominant for raw images, MLPs serve as the classification head in many image recognition pipelines. They take high-level features extracted by convolutional layers and map them to class probabilities.

9. Conclusion

An MLP is a foundational neural network consisting of input, hidden, and output layers with fully connected neurons.

Training relies on forward propagation, loss computation, backpropagation, and gradient descent to iteratively minimise prediction error.

Activation functions (ReLU, Sigmoid) introduce the non-linearity required to model complex real-world relationships.

Regularisation methods such as Dropout and L2 are essential to prevent overfitting and ensure strong generalisation.

MLPs have broad, proven applications across healthcare, finance, and computer vision, making them a critical tool in any practitioner's toolkit.

10. References

Goodfellow, I., Bengio, Y. and Courville, A. (2016) *Deep Learning*. Cambridge, MA: MIT Press.

Chollet, F. (2021) *Deep Learning with Python*. 2nd edn. Shelter Island, NY: Manning Publications.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M. and Duchesnay, É. (2011) 'Scikit-learn: Machine learning in Python', *Journal of Machine Learning Research*, 12, pp. 2825–2830.

Abadi, M., Barham, P., Chen, J., Chen, Z., Davis, A., Dean, J., Devin, M., Ghemawat, S., Irving, G., Isard, M., Kudlur, M., Levenberg, J., Monga, R., Moore, S., Murray, D.G., Steiner, B., Tucker, P., Vasudevan, V., Warden, P., Wicke, M., Yu, Y. and Zheng, X. (2016) 'TensorFlow: A system for large-scale machine learning', in *Proceedings of the 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI '16)*. Savannah, GA: USENIX Association, pp. 265–283.

UCI Machine Learning Repository (no date) *Pima Indians Diabetes Database*. Available at: https://archive.ics.uci.edu (Accessed: 27 March 2026).